

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللہی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 11/30

Introduction to Functions: Part 2

- More on argument passing and function return types
- Writing more functions
- Inline functions
- References and further readings
- Q & A

120 min (incl. Q & A)



Return type, Value return and void

- A function should return a value unless it is a void function.
- void indicates absence of information.
- There are no objects of type void.

Return type

```
int f1() { } // error: no value returned
void f2() { } // ok
int f3() { return 1; } // ok
void f4() { return 1; } // error: return value in void function
int f5() { return; } // error: return value missing
void f6() { return; } // ok
```

Value return

- void has two main applications:
 - Function: void return type
 - Generic pointer: void*
- The semantics of *function value return* are identical to the semantics of *initialization*. A return statement is considered to initialize an *unnamed variable* of the returned type.
- Variable is an object that has a name.

Object vs. Variable

Value return, some points _{cont.}

```
int* fp() // bad, don't do this
{
    int local = 1; // automatic variable
    /* ... */
    return &local; // return pointer to local
}
```

local destroyed/deallocated here

```
int& fp() // bad, don't do this
{
    int local = 1;
    /* ... */
    return local; // return reference to local
}
```

local destroyed/deallocated here

```
int fp() // good
{
    int local = 1;
    /* ... */
    return local; // OK: return a copy of local
}
```

Inline functions

- A function can be defined to be inline.
- `inline` is a function specifier. other specifiers are `virtual` and `explicit`.
- The inline specifier indicates to the implementation that inline substitution of the function body at the point of call is *to be preferred* to the usual function call mechanism.

```
// return the maximum value of two integers  
int Max(int x, int y)  
{  
    return x > y ? x : y;  
}
```

```
void f()  
{  
    int m = Max(3, 5); // function call  
}
```

Function call overhead



```
// return the maximum value of two integers  
inline int Max(int x, int y)  
{  
    return x > y ? x : y;  
}  
  
void f()  
{  
    int m = Max(3, 5); // expand inline  
}
```

Inline functions cont.

- The inline specifier is a *hint* to the compiler. Compilers are free to ignore inline directives and to make their own decisions about which functions to inline.
- An inline specifier does not affect the semantics of a function.
 - Inline function:
 - no function call overhead
 - larger program
 - faster program
 - Non-Inline function:
 - function call overhead
 - smaller program
 - slower program

Q Are inline functions guaranteed to make your performance better?

A No. Beware that overuse of inline functions can cause *code bloat*, which can in turn have a negative performance impact in paging environments.



R references and further readings

[KR78] Marshal Cline, Greg Lomow, and Mike Girou. C++ FAQs, 2nd edition, Addison-Wesley, 1998.

[KR78] Brian W. Kernighan & Dennis M. Ritchie. The C Programming Language, Prentice-Hall, Englewood Cliffs, NJ, 1978.

[Str00] Bjarne Stroustrup. The C++ Programming Language (Special Edition). Addison Wesley. Reading Mass. USA. February 2000.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

